Credit Card Fraud Binary Classification Model

Zoey Katzive
Brown University Data Science Institute
GitHub

1.0 Introduction

As digital payment methods continue to evolve, it is important for companies to protect their consumers from fraud. In 2021, over 389,000 cases of credit card fraud were reported in the United States totaling 181 million dollars in losses,[1] and credit card fraud reports for existing accounts were up +22% in 2022.[2] It is the responsibility of credit card companies to leverage machine learning methods to catch fraudulent transactions when they occur to protect their consumers.

I analyzed a simulated dataset on credit card fraud classification. Sources on credit card fraud are highly confidential, and the ones that I could find contained almost all classified features. Since my goal for this project is an academic one, working with unredacted features seemed more constructive.

The dataset contains four categorical features: used_chip, used_pin , online_order, and repeat_retailer, and four continuous features: distance from home, ratio to median purchase price and distance from last transaction. The categorical features are all classified with zeroes and ones, with one signifying "yes" and zero signifying "no". The target variable, "fraud", is classified similarly, with an input of one signifying that the given transaction is fraudulent. The dataset contains one million observations.

Two limitations of this dataset is that its description does not contain units and there are only seven non-target features which is not a realistic representation of how companies detect fraud.

The dataset is from Kaggle[3], and there are eighty publicly available notebooks of previous work. Several contributors achieved nearly 100% accuracy with a small number of false positives and negatives on their test sets. One contributor used GridSearchCV with a non-regularized logistic regression model and achieved an accuracy of 99% with very few false

[1] https://money.usnews.com/credit-cards/articles/what-is-credit-card-fraud
[2] https://www.washingtonpost.com/technology/2023/08/02/credit-card-fraud-tips-ai/
[3] https://www.kaggle.com/datasets/dhanushnarayananr/credit-card-fraud

positives.[4] Another user trained a Logistic Regression, a KNearestNeighbors classifier and a Random Forest classifier, and found that the Random Forest model yielded the best results, generating just 3 false positives and zero false negatives with an accuracy of 99.9%.

My model performed similarly to these others. I achieved 99.9% accuracy with a Random Forest classifier, zero false positives or negatives with a test set of size of 2% and one false negative and two false positives with a test set size of 20%.

2.0 Exploratory Data Analysis

My first step in the exploratory data analysis (EDA) process was to check for missing values, of which there are none in my dataset. I then checked the value counts of each feature to get a better understanding of the data I was working with. After these steps, I moved on to visualization.

I visualized the target variable by creating a pie chart that depicted the percentage of transactions that were classified as fraudulent.

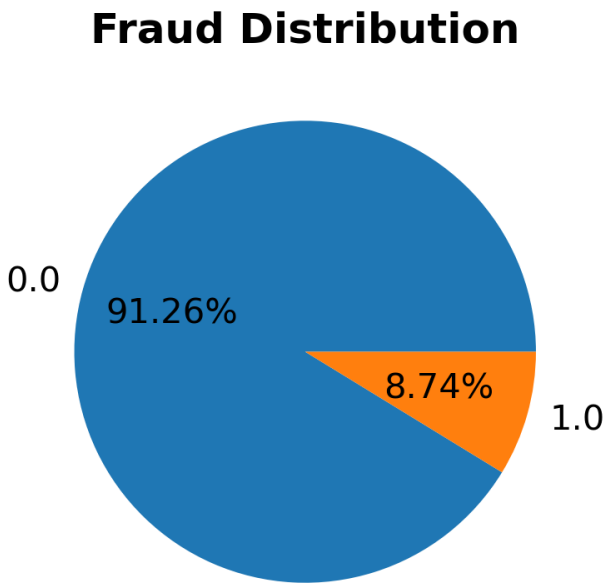

Figure 2.1: 8.74% of transactions in the dataset are fraudulent.

[4] https://www.kaggle.com/code/ahmedmohammedhamada/credit-card-fraud-detection-with-accuracy-99-9

Less than 9% of all transactions in the dataset are classified as fraudulent. This is important to note when it comes to splitting the data, as it is imbalanced. I knew here that to prevent creating a test set that only contains observations from class zero, I would need to create a stratified split later on.

I then looked at the distributions of the categorical variables, which will help me contextualize my results later on.

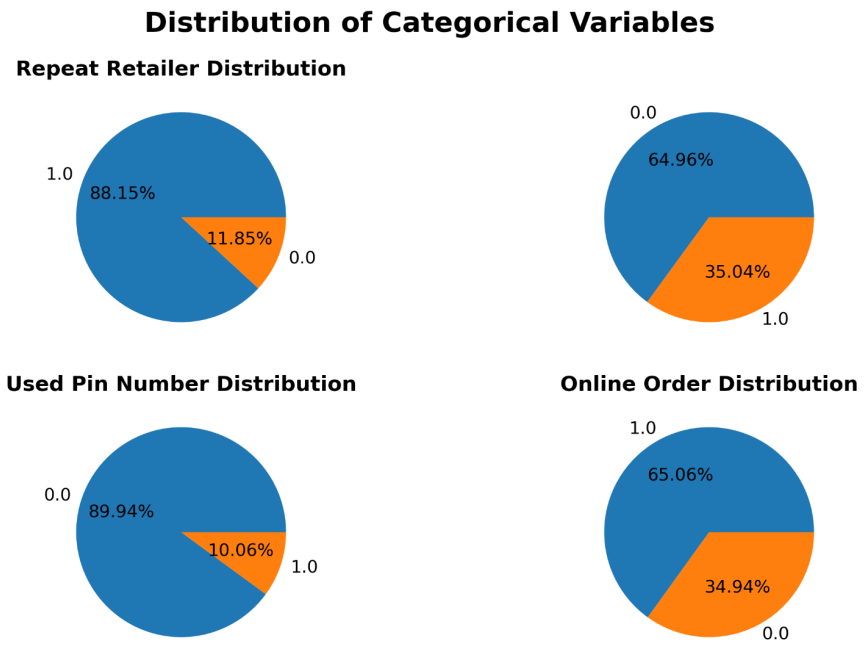

Figure 2.2: Distributions of the categorical variables, with (1) meaning “yes” and (0) meaning “no”

Next, I examined how the categorical features correlated with the target variable. Significantly, I found that nearly every single transaction in which a pin number was used was non-fraudulent, which suggests that security measures by credit card companies are working as intended. We also see that significantly more fraudulent transactions occurred for online orders, which is consistent with the real-world data that has come out in recent years. As more people shop online, more credit card fraud occurs.

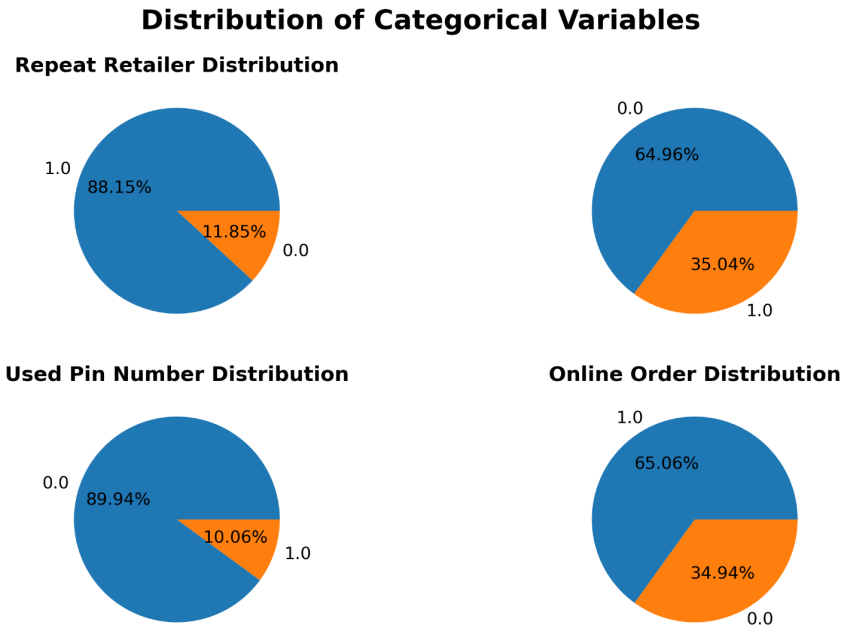

Figure 2.3: Relationship between categorical variables and the target. Significantly more fraud occurred for online orders and less occurred when pin numbers were used.

I next visualized each continuous variable's relationship with the target by creating histograms. I found that transactions that had a higher ratio to the user's median purchase price were much more likely to be fraudulent, which makes logical sense since the spending habits of fraudsters are unlikely to mirror those of their victims. Fraudulent transactions were also more likely to occur at further distances from the user's home and their last transaction.

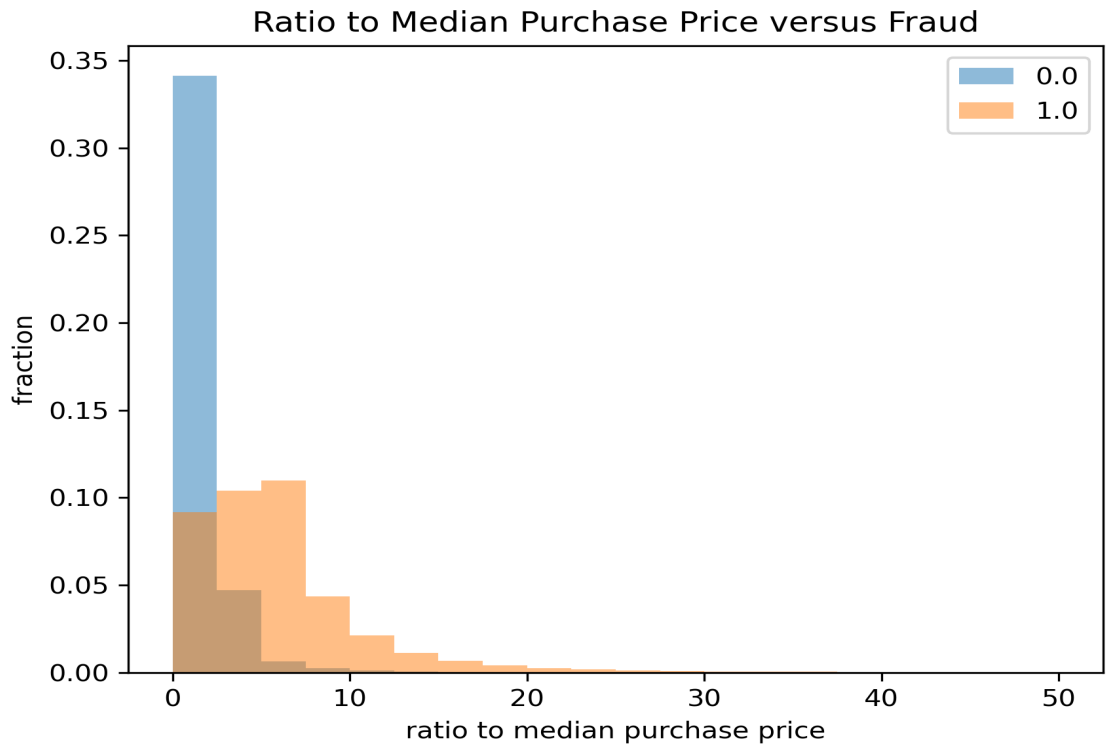

Figure 2.4: Fraudulent transactions are more common when the ratio to the user's median purchase price is further from 1.

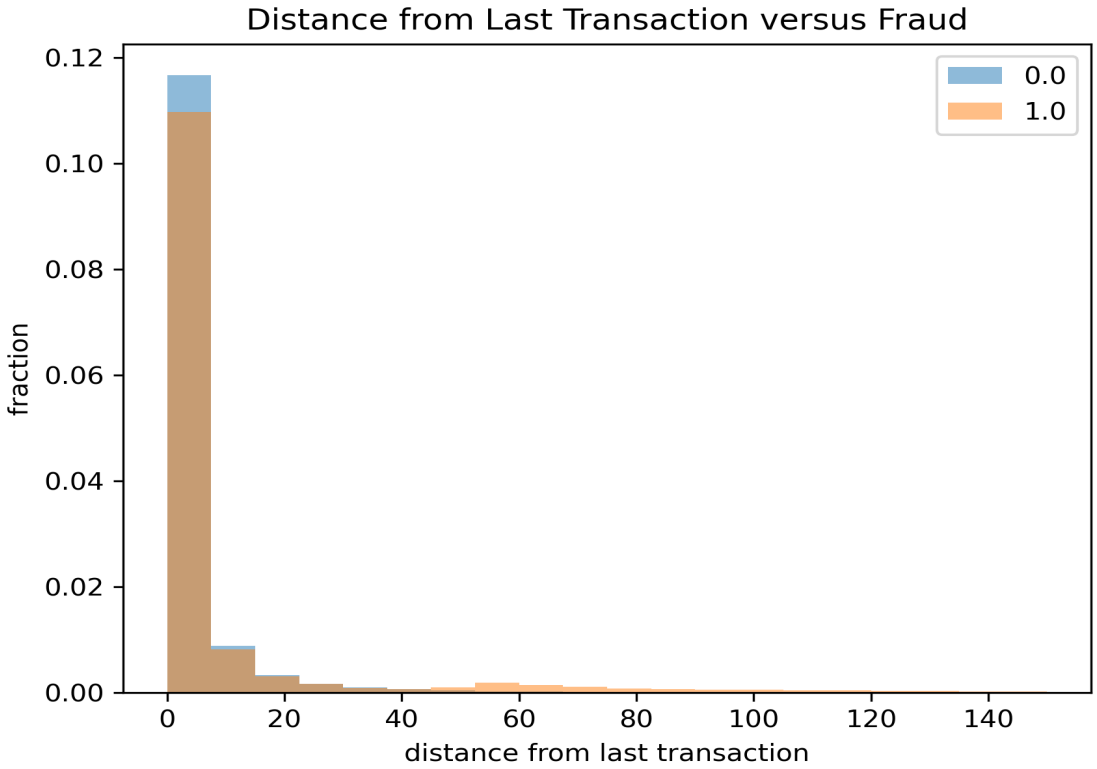

Figure 2.5: Fraudulent transactions are more likely to occur at further distances from the user's previous transaction.

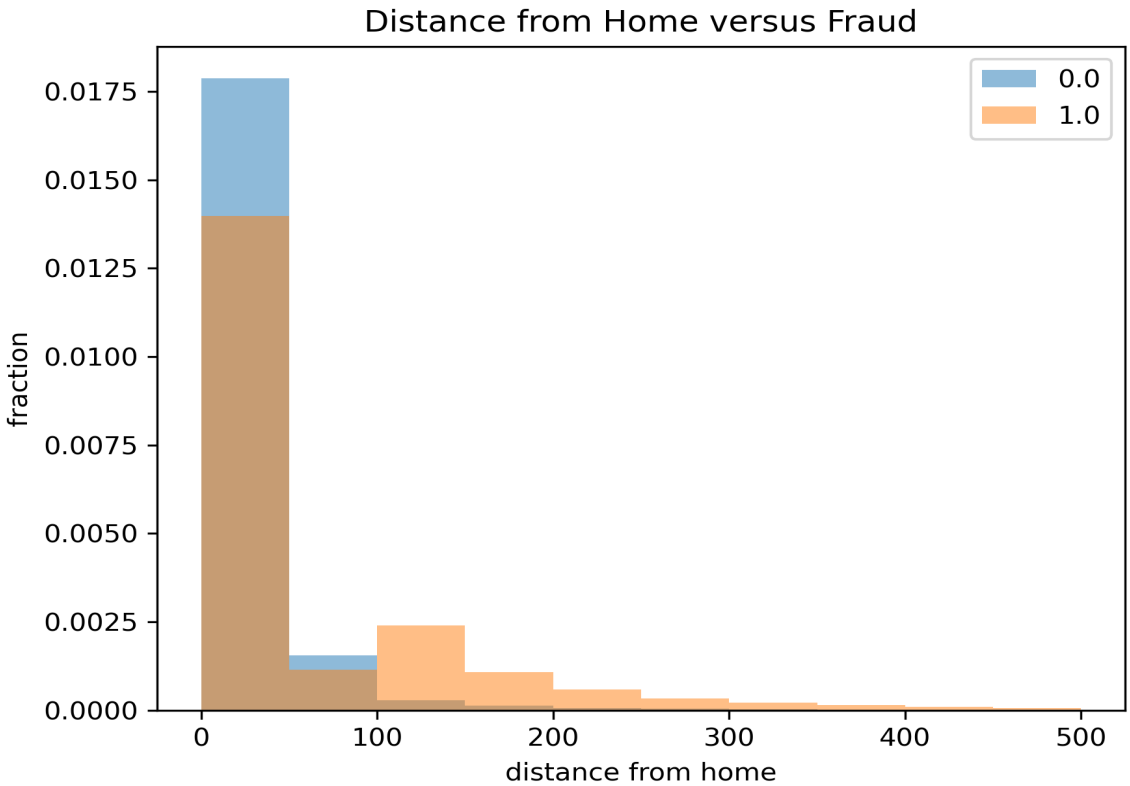

Figure 2.6: Fraudulent transactions are more common at further distances from the user's home.

Finally, I made a heatmap of the correlation matrix between the continuous features and the target. We can see that the highest-correlated feature is ratio to median purchase price, which has a positive correlation coefficient with the target of 0.46. Because this dataset is simulated, it is interesting that no variable is completely correlated with the target, as that

would have explained our high accuracy later on.

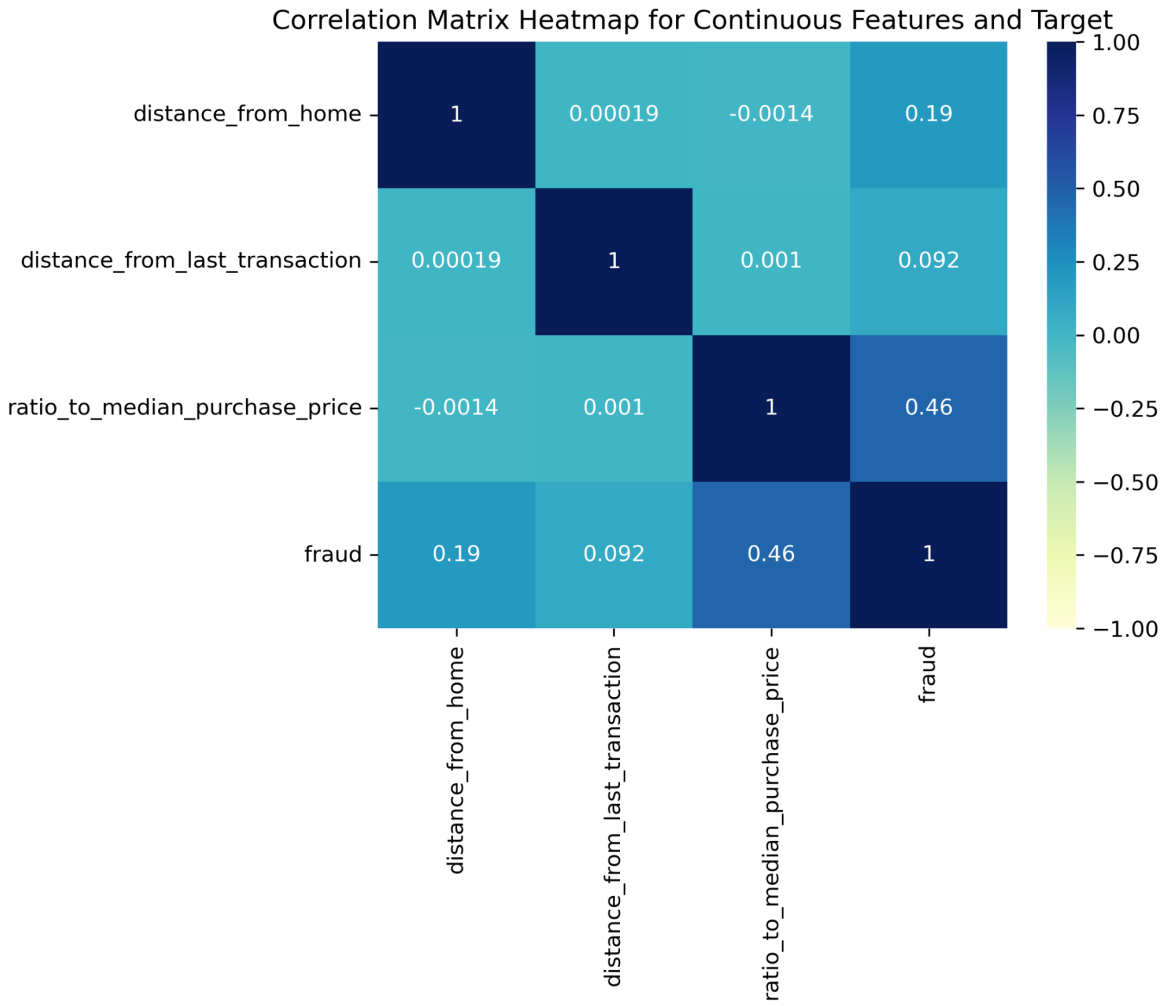

Figure 2.7: This heatmap shows the correlation coefficients between each continuous feature and the target.

3.0 Methods

I used a stratified basic split to create the test set and then a stratified K-fold for cross validation. This ensures that neither the test nor validation sets consist entirely of observations from class 0, as my dataset is imbalance.

To preprocess, I used OneHotEncoder for the categorical features and StandardScaler for the continuous ones. Since there are no units attached to the continuous variables, standardizing these features was crucial. I chose to use StandardScaler rather than MinMaxScaler as each of the continuous features followed a tail distribution, and I ultimately

used ColumnTransformer to accumulate both transformers into one variable that I could plug into my pipeline later.

To create my pipeline, I wrote a function that takes in the feature matrix, the target variable, a random state, and the number of folds for the K-fold cross validation step. Within this function, I split the data, set the preprocessors and the parameter grid for each model I tested, and then fit the model using GridSearchCV to improve my cross validation. Outside of the function, I wrote code that returns the accuracy and the parameters that yielded the highest scores. Overall, this function made it easy to test different models and parameters as needed.

I trained four models with this pipeline to test which performed at the highest accuracy. The algorithms I used were Logistic Regression, Random Forest Classifier, KNearestNeighbors Classifier, and SVC. For my logistic regression parameters, I used seven uniformly spaced alpha values in log between le-3 and le3. For the decision tree, I played around with different numbers of estimators and maximum depths. For the nearest neighbors, I used four different numbers of neighbors, and for the SVC I applied the same logspace as for the logistic regression and used two different gamma values, “scale” and “auto”. The Random Forest classifier turned out to be the most predictive.

To measure the uncertainty of my evaluation metric, I ran the Random Forest model on two different test sets with sizes 0.02 and 0.2. Since I had such a low number of false positives and negatives on my smaller test size, I ran it on the larger test set to see how that would change. Surprisingly, the accuracy score stayed almost the same, and the number of false results increased minimally.

4.0 Results

The baseline accuracy score is the fraction of true positives in the dataset, which is 0.912, and each model performed at a higher accuracy than this. The Random Forest classifier outdid the rest in terms of generating minimal false results, as it only yielded two false positives and one false negative at a critical probability of 0.25. It is likely that the simulated nature of the dataset allowed the decision tree classifier to perform extremely well.

Model	Parameters	Accuracy Score	False Positives	False Negatives
Logistic Regression	np.logspace(-3, 3, 7)	0.956	4,625	3,273
KNearestNeighbors	n_neighbors: [1, 3, 5, 10]	0.993	123	137
Random Forest	n_estimators: [10, 50, 100], max_depth': [10, 20]	0.999	2	1
SVC	svc__C: np.logspace(-3, 3, 7), svc__gamma: ['scale', 'auto']	0.995	17*	3*

Table 4.1: Parameters, accuracy score, and false positives and negatives evaluated test sets with size equal to 0.2 at a critical probability of 0.25. *The SVC model could only run on a subset of 15,000 observations with a test size of 3,000.

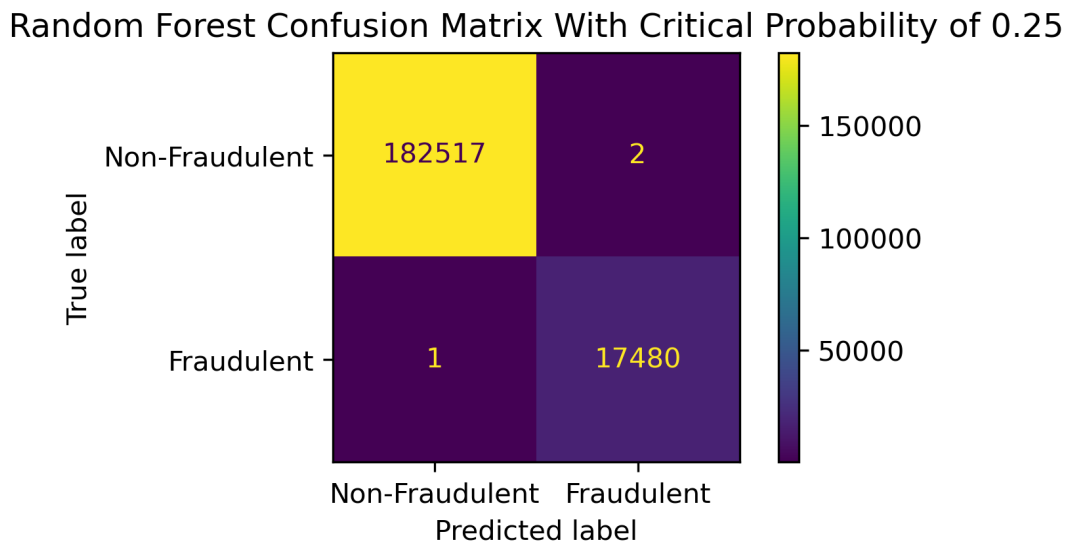

Figure 4.1: The Random Forest model only results in two false positives and one false negatives. Recall is optimized.

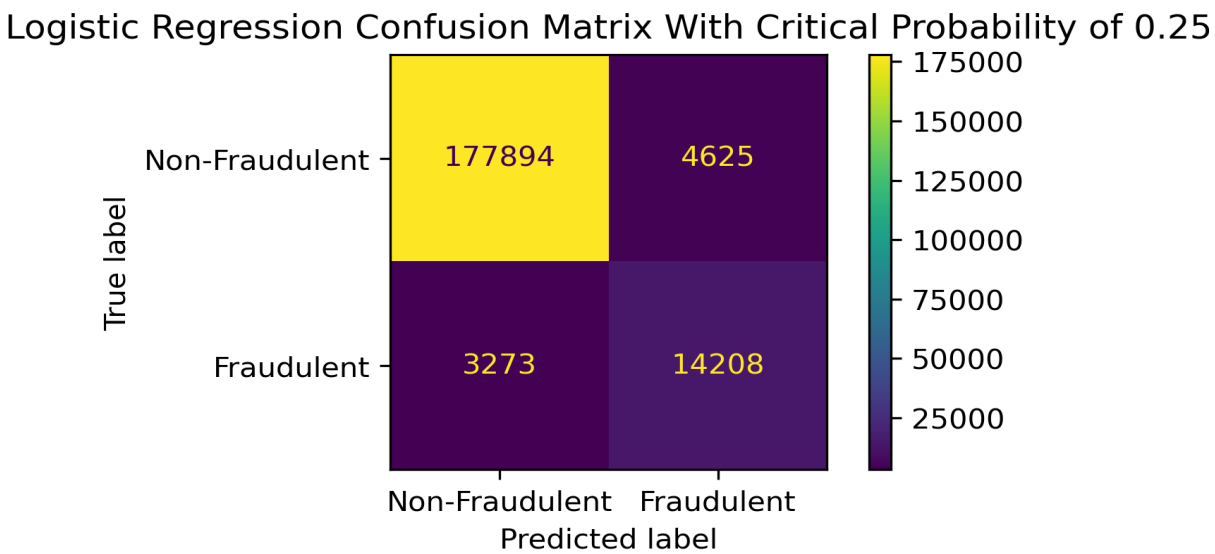

Figure 4.2: The logistic regression model yields several thousand false positives and negatives.

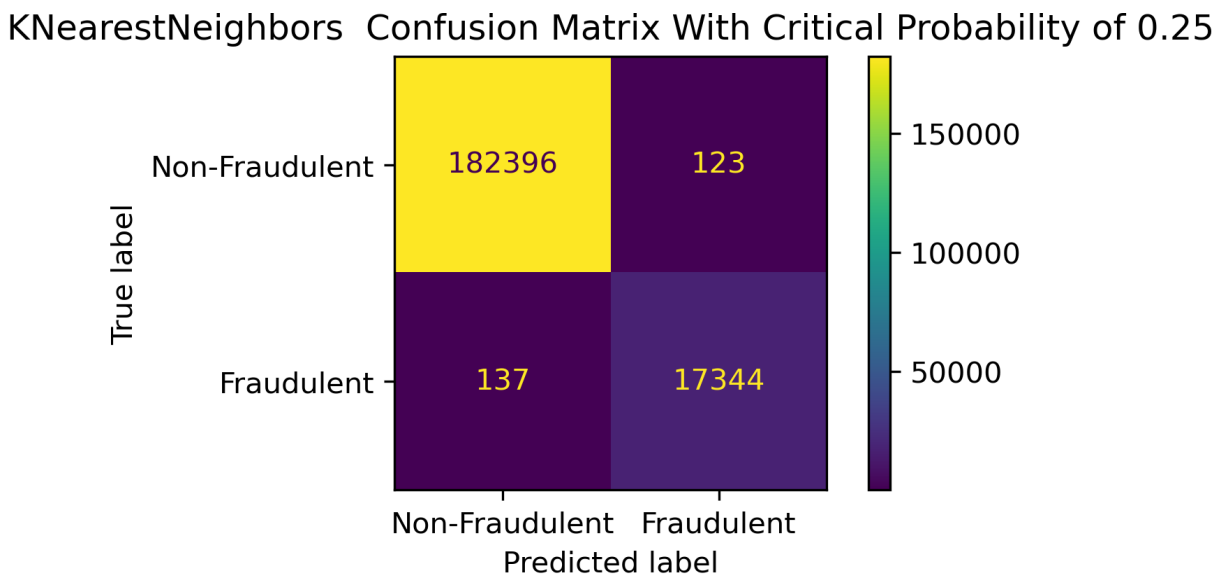

Figure 4.3: The KNearestNeighbors classifier results in a couple hundred false positives and negatives.

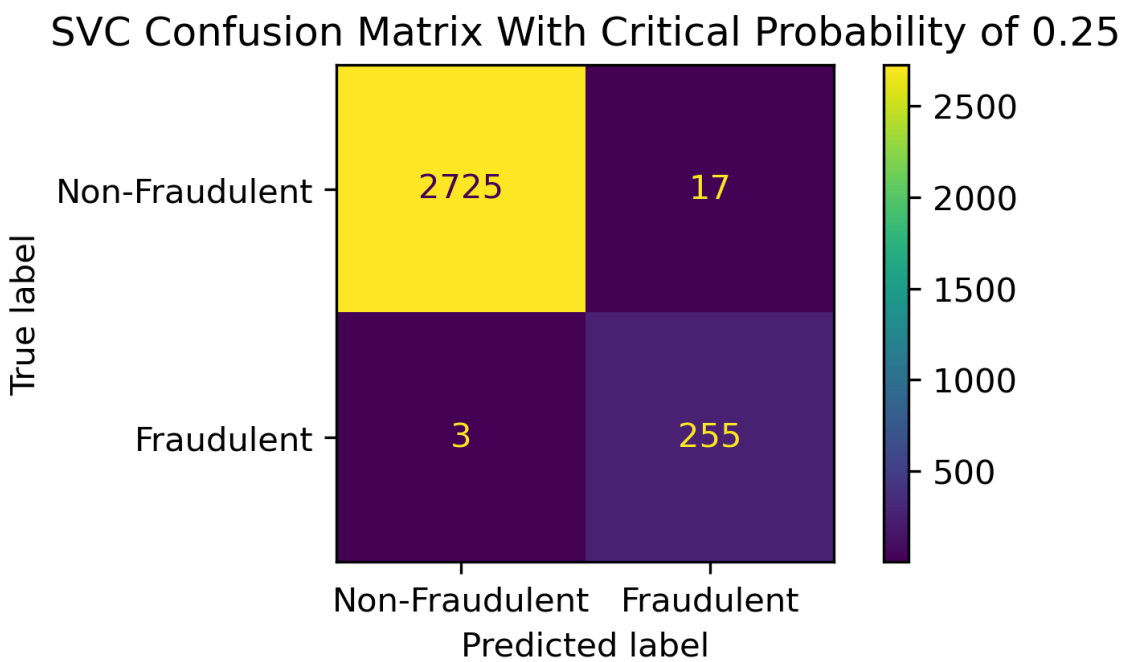

Figure 4.4: The SVC model ran on a subset of 15,000 observations, but still returned more false results than the Random Forest model.

When interpreting these confusion matrices in the context of credit card fraud, it is important to maximize recall. False positives have a minimal real-world impact as they can be

resolved through a simple phone call. False negatives, however, will cause consumers to lose money and are more consequential.

Next, to calculate global feature importances, I worked with sci-kit learn's "permutation_importance". I found that "ratio to median purchase price" had the highest impact on the accuracy score and "repeat retailer" had the lowest. This corresponded to what I learned from the correlation matrix I created in the EDA stage.

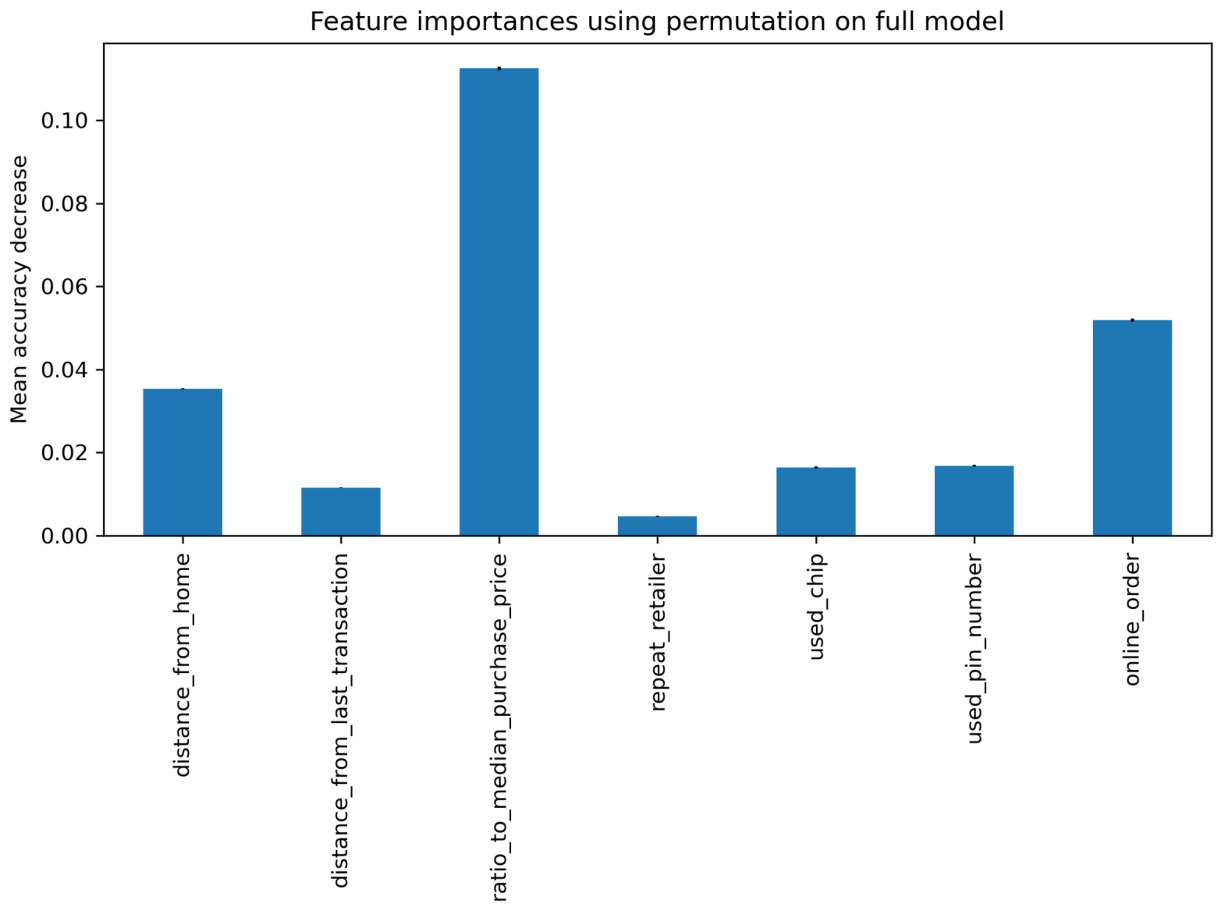

Figure 4.4: Ratio to median purchase price has the highest bearing on model accuracy.

Finally, I calculated the SHAP values of my model. Ratio to median purchase price impacted the model output the most globally, and had the highest impact on all three local indices analyzed. The first summary plot depicts each feature's average impact on model performance, and the second is a summary plot of the label [0]. The force plots represent local feature importance for two indices.

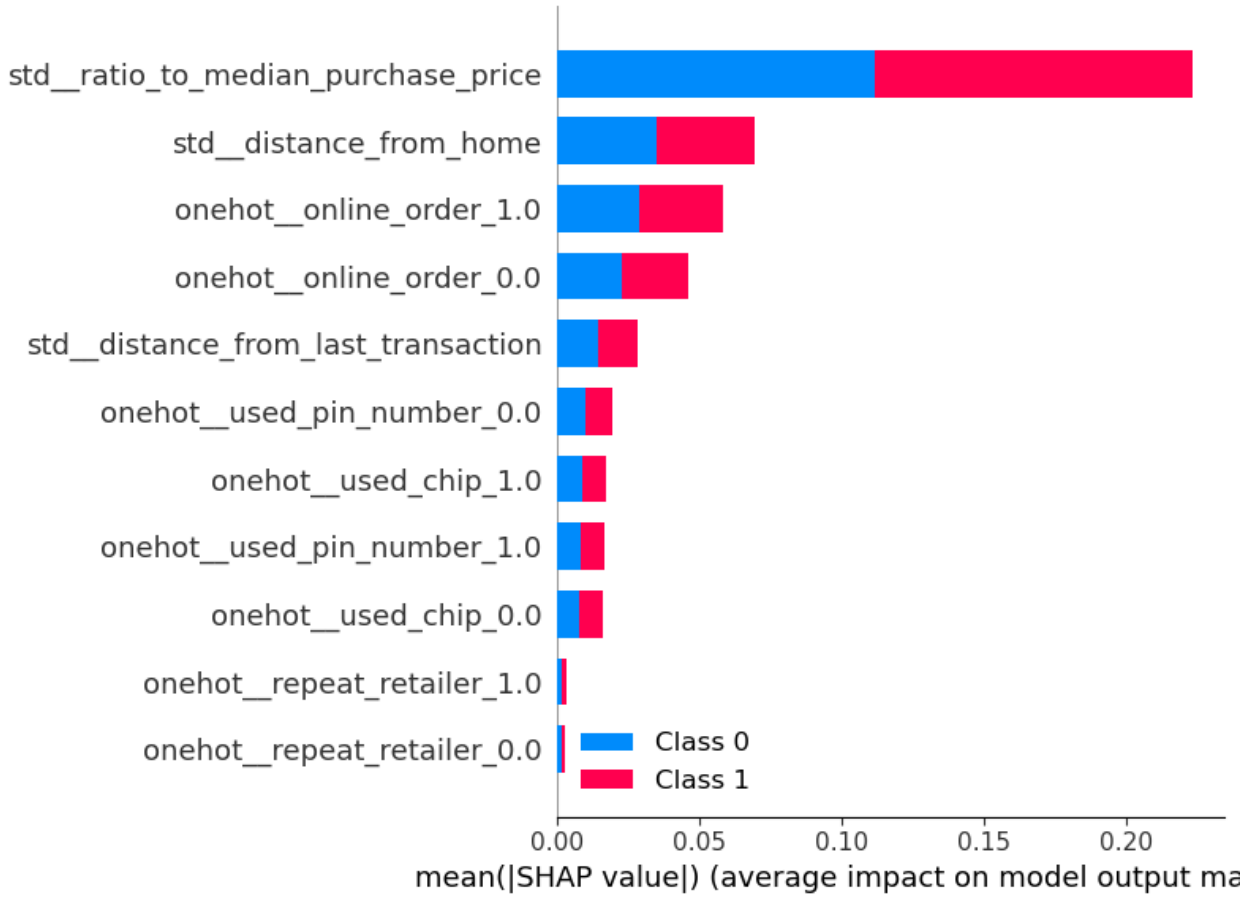

Figure 4.5: SHAP summary plot: ratio to median purchase price has the highest mean SHAP score.

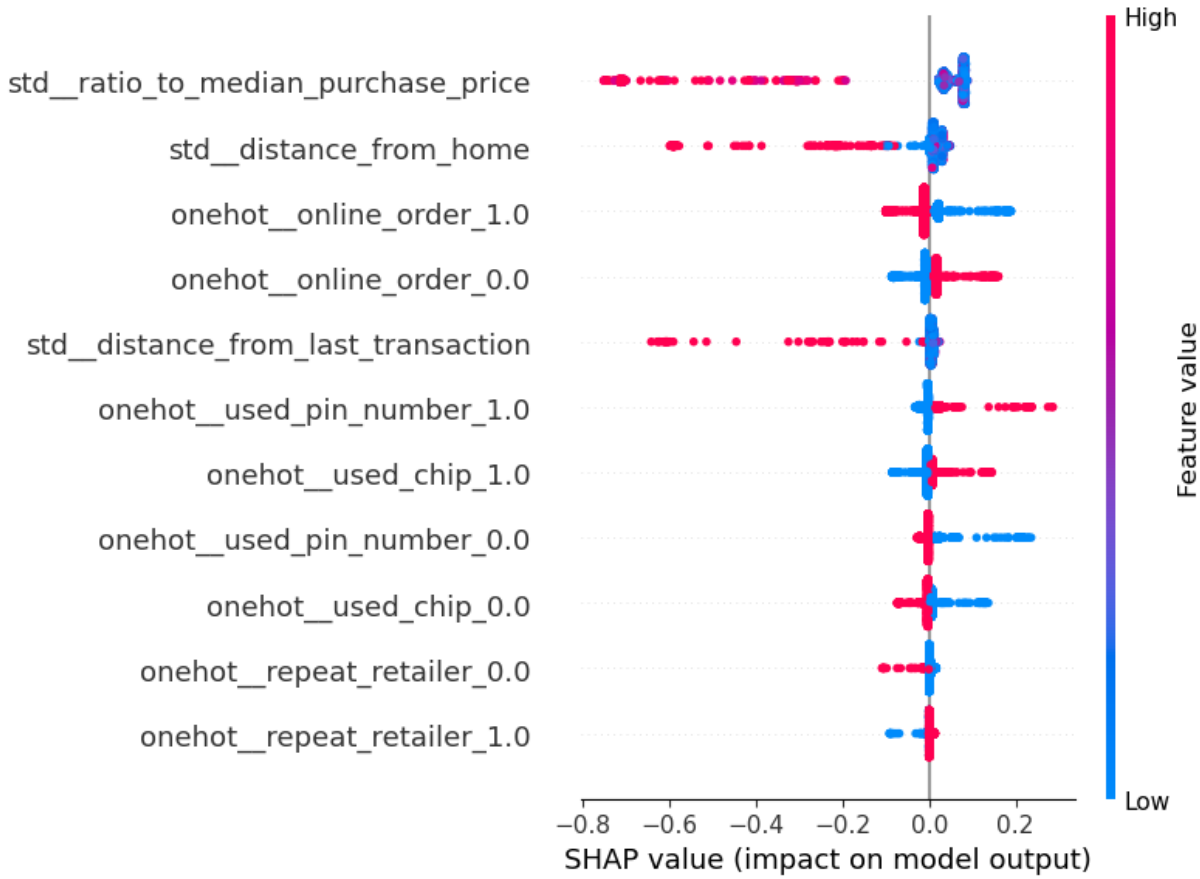

Figure 4.6: SHAP Summary plot: ratio to median purchase price and distance from home and last transaction highly impact model output.

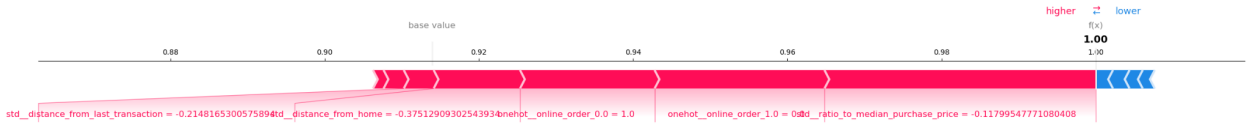

Figure 4.7: Force plot of local SHAP values for index 0. Ratio to median purchase price has the highest positive impact on prediction.

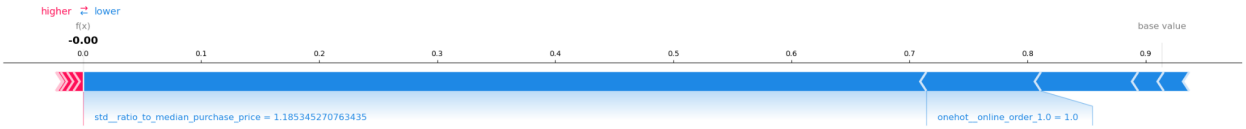

Figure 4.8: Force plot of local SHAP values for index 14. Ratio to median purchase price has the highest negative impact on prediction.

While the dataset is simulated, it is still interesting to see how strong the predictive power of this model is when the highest correlation coefficient between a feature and the target is just 0.46. It is unsurprising that ratio to median purchase has the highest impact on the model's prediction, given the context that scammers will typically out-spend their victims, but it is impressive that a decision tree model classified nearly every single point on the test set correctly.

5.0 Outlook

While the high accuracy of this model on simulated data does not necessitate that real-world models will perform as well, this project proves that machine learning is extremely useful in the context of credit card fraud. With more time, I would have trained an XGBoost model to calculate more global feature importances, and I would have run the SVC model on the entire dataset. Most importantly, I would have tuned more hyperparameters for the Random Forest model to see how it would impact its accuracy. It would be very useful to have a stronger sense of which parameters give the very best result if we were to apply this model to real-world data. Overall, I am optimistic about the high potential of utilizing machine learning methods to keep consumers safe.

6.0 References

1. Williams, Geoff. "What Is Credit Card Fraud?" U.S. News & World Report. Jan. 13, 2023 https://money.usnews.com/credit-cards/articles/what-is-credit-card-fraud.

2. Albergotti, Reed. "These tips from the experts could help you avoid credit card fraud, but they’re no guarantee." The Washington Post, August 2, 2023. Accessed [Access Date]. https://www.washingtonpost.com/technology/2023/08/02/credit-card-fraud-tips-ai/.
3. Dhanush Narayanan R. "Credit Card Fraud." Kaggle. https://www.kaggle.com/dhanushnarayananr/credit-card-fraud
4. Hamada, Ahmed Mohammed. "Credit Card Fraud Detection with Accuracy 99.9%." Kaggle. https://www.kaggle.com/code/ahmedmohammedhamada/credit-card-fraud-detection-with-accuracy-99-9.